Empowering Java Development with Agent Skills

SPEAKER

Tristan Mahinay

Tristan Mahinay

Solutions Architect & Open-Source Committer



Solutions Architect and Java Practice Leader at IBM focusing on Application Design and Architecture.

Community Leader for the Java User Group Philippines (JUG PH) and Apache Kafka Manila

Open-Source Lead Maintainer for IBM and Spring AI Community

@rjtmahinay

linkedin.com/in/rjtmahinay

linktr.ee/rjtmahinay

facebook.com/rjtmahinay

Agent Skills Replace Fragile Prompts

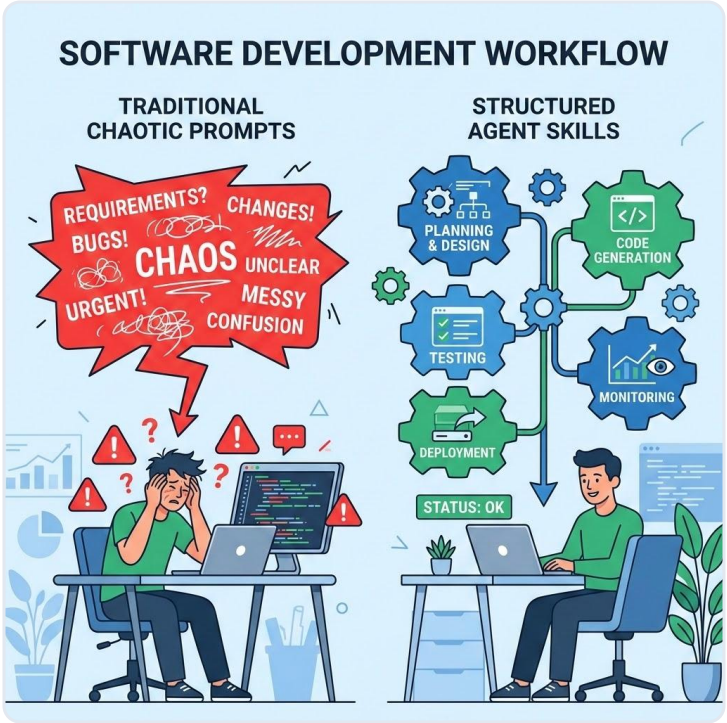
with Reliable Systemic Tools

Fragile Traditional Prompts

Transitioning away from copying and pasting lengthy, one-off prompts into chatbots.

Structured Agent Skills

- Agent Skills act as declarative tools that package reusable workflows and domain expertise.
- They provide defined inputs, predictable outputs, and structured execution.
- This shift ensures that the same instructions execute consistently across every team member and platform.



Worst-Case Scenario: The Mega-Prompt Pitfall

Why Relying on Unstructured Long Prompts Fails at Scale

1. Context Window Degradation & High Costs

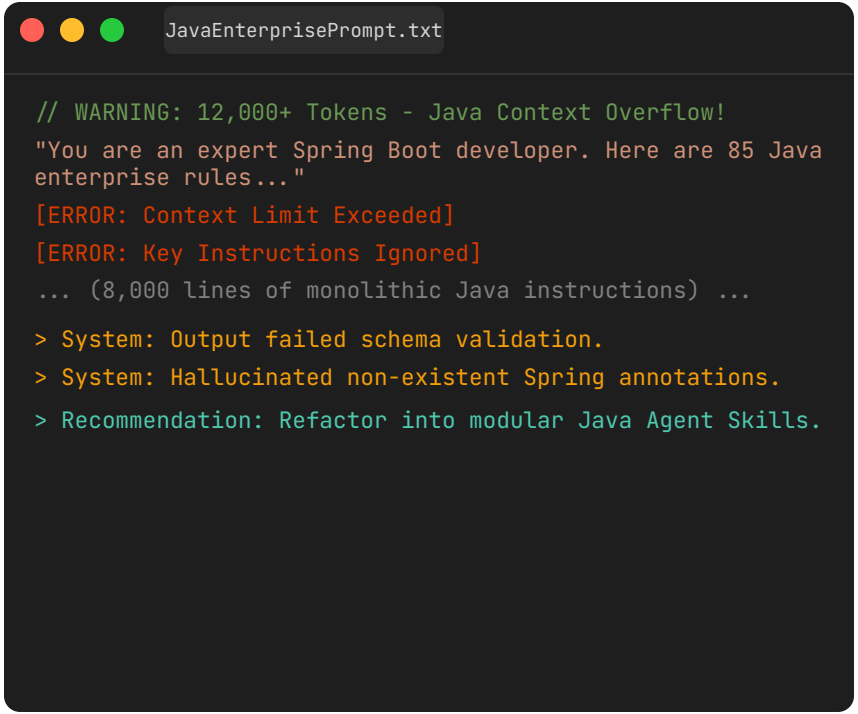
- Massive prompt overhead re-sent on every query spikes token costs exponentially.
- Crucial instructions get lost in the middle ("Lost in the Middle" phenomenon).

2. Non-Deterministic & Unreliable Execution

- Unstructured prompts lead to unpredictable hallucinations and missing steps.
- No guaranteed schema validation or strict structural enforcement.

3. Zero Reusability & Operational Chaos

- Team members copy-paste divergent versions, causing operational drift.
- Updating workflows requires manual edits across hundreds of user sessions.



Decomposing the Mega-Prompt Monolith

Refactoring 12,000+ Unstructured Tokens into Modular Agent Skills

JavaEnterprisePrompt.txt

```
// MONOLITH MEGA-PROMPT
// 12,000+ Unstructured Tokens

[Section 1] Domain Context
• Embedded enterprise guidelines
• Context window bloat

[Section 2] Syntax & Rules
• Unenforced JSON schema
• Frequent hallucinations

[Section 3] Workflow Governance
• Hardcoded instructions
• Difficult to maintain

... 8,000+ monolithic lines ...
```

Decompos

01. Context & Cost Skill

Optimized Context Window

- Encapsulates specific domain context.
- Reduces token payload by loading on-demand.
- Prevents "Lost in the Middle" errors.

02. Deterministic Validation Skill

Strict Schema & Syntax Rules

- Enforces rigid output schemas.
- Eliminates non-existent annotations.
- Guarantees reliable code generation.

03. Modular Governance Skill

Centralized & Reusable

- Single source of truth for engineering teams.
- Zero copy-pasting; seamlessly version-controlled.
- Instantly updates across user sessions.

SKILL.md

```
# Modular Agent Skills Package
---
📁 skills/
├── 📁 01_context_cost/
│   └── SKILL.md
├── 📁 02_schema_validator/
│   └── SKILL.md
└── 📁 03_modular_governance/
    └── SKILL.md
---
> Status: 3 Isolated Modular Skills
> Execution: 100% Deterministic
> Payload: -85% Token Reduction
// Why -85%? Progressive disclosure loads minimal
catalog metadata (~50 tokens/skill) at startup and
expands full instructions on-demand instead of
keeping all workflows permanently loaded in context.
```

agentskills.io Provides a Standard Specification for AI Capabilities

- Agent Skills are a lightweight, open format for extending AI capabilities.
- A skill is a simple directory containing a SKILL.md file.
- The file includes YAML frontmatter for metadata (name, description) and markdown for instructions.
- Originally created by Anthropic, it is now an open standard adopted by platforms like Gemini CLI and GitHub Copilot.

SKILL.md

```
# SKILL.md Frontmatter Example
---
name: "web_search"
description: "Search the web for information"
---

# Web Search Skill

This skill allows the AI to query global knowledge
bases in real-time.

## Instructions

1. Parse keywords from the prompt.
2. Execute the prioritized search API call.
```

Anatomy & Core Components of an Agent Skill

Extend model capabilities with structured instructions, code, assets, and documentation.



1. SKILL.md (Entrypoint)

Required Manifest & Instruction Set

- `Frontmatter`: Defines name and description for JIT discovery.
- `Body`: Detailed instructions, rules, and operational constraints.
- Serves as the primary prompt interface for the LLM agent.

2. Scripts Directory

Executable Code & Automation

- Contains Python, Bash, or Node.js executables for complex tasks.
- Offloads deterministic computation from the model context.
- Executed directly by the agent runtime environment on demand.

3. Assets Directory

Templates, Schemas & Resources

- Holds static files, boilerplate code, JSON schemas, and images.
- Reusable visual or textual components injected into outputs.
- Maintains output style, brand, and layout consistency.

4. References Directory

On-Demand Context & Documentation

- In-depth domain docs loaded via progressive disclosure.
- Prevents main prompt bloat by expanding details only when needed.
- Includes specs, extended rules, and reference manuals.

Best Practices for Designing High-Performance Agent Skills

Build robust, context-efficient AI workflows by packaging domain expertise into modular SKILL.md structures.

01

Spend Context Wisely

› Keep the main `SKILL.md` file under 500 lines to preserve the context window.

› Focus exclusively on what the agent lacks and omit what it already knows.

› Utilize a three-stage progressive disclosure strategy to load files on demand.

02

Calibrate Control

› Match the level of instruction specificity to the inherent fragility of the task.

› Rely on third-person imperative commands to direct precise agent behavior.

› Embed structured templates and deterministic validation loops for consistent outputs.

03

Optimize Discovery

› Draft concise, intent-focused skill descriptions under the 1024-character limit.

› Front-load descriptions with essential trigger words to ensure reliable activation.

› Set up a systematic evaluation loop to validate triggers and avoid overfitting.

Optimize Descriptions to Load Right Agent Skills on Demand

Descriptions drive semantic matching and JIT activation to manage context footprints.

Principles of Description Design

- **Use Imperative Phrasing:** Frame descriptions as "Use this skill when..." rather than passive declarations.
- **Focus on User Intent:** Align with what user wants to achieve, not the internal implementation details.
- **Keep It Concise:** Keep descriptions short and focused (specification enforces a hard 1024-character limit).
- **Avoid Overfitting:** Design triggers with broad, generalized semantic categories instead of raw keywords.

Java Skill Curation Example

`spring-data-jpa/SKILL.md`

Trigger Intent: When implementing paginated Spring REST endpoints or optimizing database queries.

```
# YAML Metadata
name: spring-data-jpa-tuning
description: "Use this skill when optimizing paginated
REST endpoints to avoid N+1 and in-memory heap pagination
issues."
# Instructions
- Load parent IDs first via pagination.
- JOIN FETCH child collections next.
```

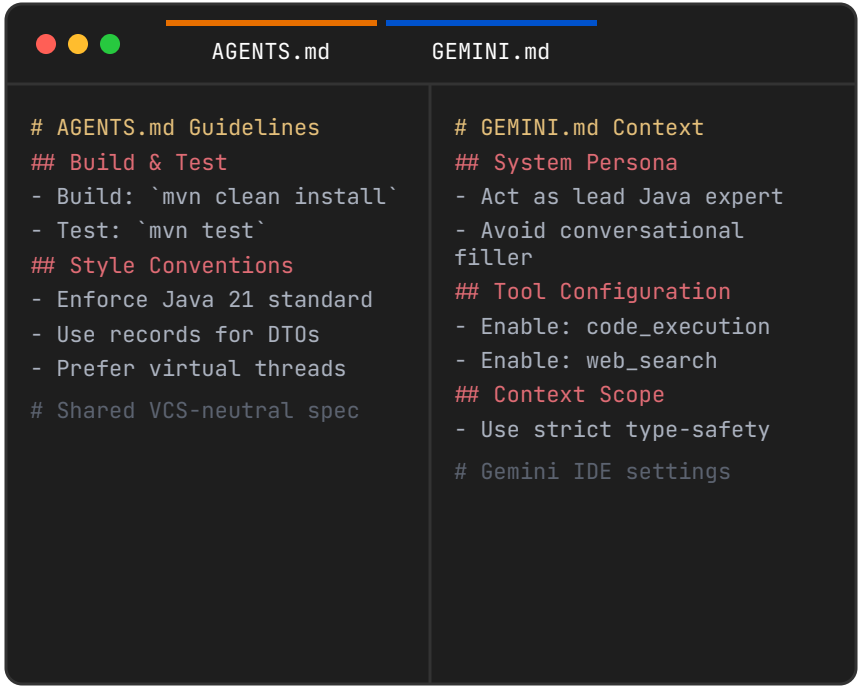
Progressive Disclosure Optimizes the Agent Context Window

- Agents load only the skill's name and description at startup, consuming roughly **30 to 50 tokens** per skill.
- The full `SKILL.md` instructions load into the context window only when triggered by a matching task.
- Referenced files, scripts, and assets are loaded **dynamically** during execution.
- This prevents **token bloat**, allowing hundreds of skills to be installed without degrading performance.



Standardize Environments Using AGENTS.md and GEMINI.md

- **AGENTS.md** establishes the general baseline instructions and behavioral boundaries for the AI agent.
- **GEMINI.md** is utilized for Gemini-specific system prompts and specialized tool definitions.
- This configuration standardizes the execution environment across the team.
- It ensures the agent has the precise context needed to execute complex enterprise Java workflows reliably.



Agent Skills Automatically Enforce Modern Java 21 Best Practices

Automated Code Refactoring

Configure skills to automatically review and refactor legacy code to modern standards.

Record Types for Immutability

Enforce the use of Record types for clean, safe, and immutable data structures.

Pattern Matching for instanceof

Apply Pattern Matching for instanceof to drastically reduce boilerplate casting.

Sealed Classes & Switch Expression

Utilize Sealed Classes for strict domain modeling and exhaustive pattern-matching in switch expressions.

Java21BestPractices.java

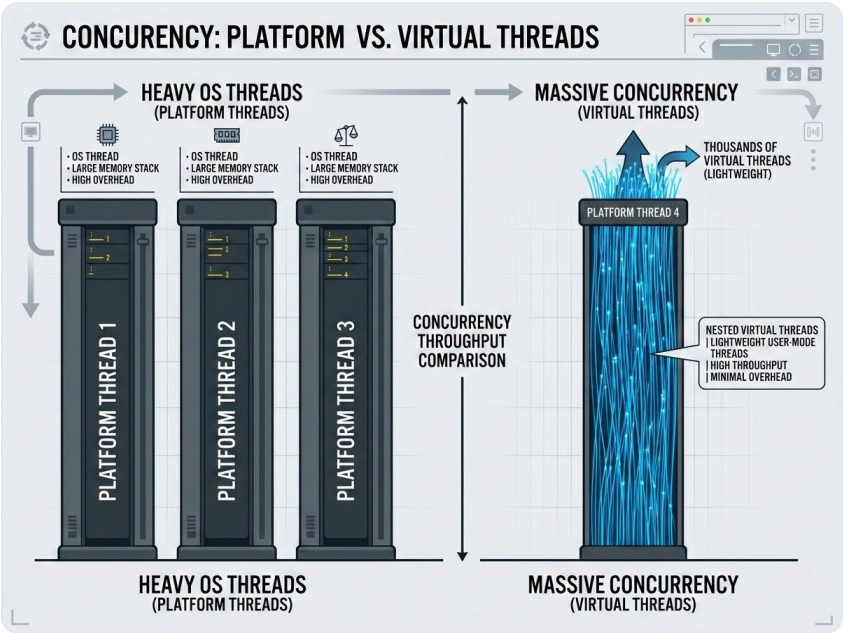
```java
// 1. Enforce Immutable Record Type
public record Point(int x, int y) {}

// 2. Pattern Match + Switch Expression
public String describe(Object obj) {
return switch (obj) {
case Point(int x, int y) when x > 100
-> "Far point at (" + x + ")";
case Point(int x, int y)
-> "Standard point";
case String s
-> "String of len: " + s.length();
default
-> "Unknown object structure";
};
}
```

Automate Performance Tuning

with Java 21 Virtual Threads

- Define specialized skills that analyze code for thread blocking and performance bottlenecks.
- The agent can automatically refactor traditional platform threads to utilize Java 21 Virtual Threads.
- Implement Structured Concurrency to manage multiple concurrent tasks cleanly.
- This optimizes application throughput and resource utilization for high-concurrency environments.



Standardize Spring Boot 4.x REST APIs

WITH BUILT-IN RESILIENCY

CORE STANDARDS

- Agent skills enforce strict REST API design standards and Spring Boot 4.x configurations.
- Automatically generate standardized controllers, services, and error handling.
- Integrate Resilience4j seamlessly into microservices.
- The agent generates the necessary boilerplate for circuit breakers, rate limiters, and bulkheads to ensure fault tolerance.

PaymentController.java

```java
@RestController
@RequestMapping("/payments")
public class PayController {

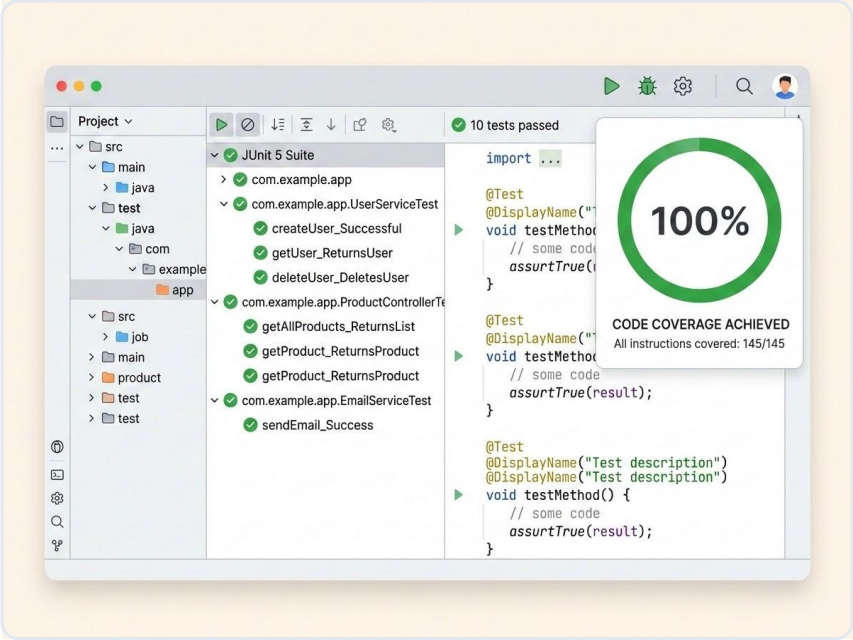
@GetMapping("/{id}")
@CircuitBreaker(name = "payCB", fallbackMethod =
"hold")
public Response pay(@PathVariable Long id) {
return service.process(id);
}

public Response hold(Throwable t) {
return new FallbackResponse("Limit Exceeded");
}

}
```

Accelerate Testing by Auto-Generating Comprehensive JUnit 5 Suites

- A **dedicated testing skill** defines the exact procedures for writing unit tests.
- The agent automatically generates robust **JUnit 5 test cases** for your business logic.
- It accurately mocks external dependencies using **Mockito**.
- The skill ensures the agent asserts complex edge cases, driving up **code coverage and reliability** without manual effort.



EXECUTABLE AGENT SKILLS

Standardized Capabilities for Java & Spring Boot Development

`/java-code-review`

Automated PR & Quality Audits

Enforces clean code, SOLID principles, and proper architectural boundaries while analyzing changesets.

`/java-security-audit`

Vulnerability Detection & Fixes

Scans for SQL injection, command injections, and common vulnerabilities, automatically drafting safe remediations.

`/java-test-engineering`

Test Suite & TDD Automation

Generates unit and integration tests using Mockito and Testcontainers, ensuring strict coverage metrics.

`/java-perf-diagnostic`

Performance & Database Optimizer

Detects memory leaks, unoptimized streams, transaction anomalies, and costly N+1 query patterns in Hibernate/JPA.

`/openshift-spring-boot-scaffolding`

Cloud-Native Bootstrapping & Setup

Creates production-ready Spring Boot applications pre-configured for OpenShift deployment, including Dockerfiles and K8s manifests.

LIVE ACTION DEMONSTRATION

See Agent Skills in Action with Java & Spring Boot

`/agent-skills-execution`

Real-Time Scenario Walkthrough

Observe how the AI agent triggers, loads, and executes complex Java workflows dynamically using standard metadata and instruction guidelines:

- **Dynamic Triggering:** Context-aware routing based on intent
- **Deterministic Run:** Precision scripts running Java quality audits
- **Self-Correction:** Validation loops detecting and fixing code failures

Extend Agents to Handle Enterprise Security and Migrations

Enterprise Spring Boot skills handle industry-standard operational requirements.



Enterprise Security

› Agents can automatically scaffold robust Spring Security configurations.

› Production-ready protection tailored to enterprise policies.



Actuator Health

› Expose Spring Boot Actuator endpoints for centralized system monitoring.

› Standardized health checks aligned with cluster orchestration tools.



DB Migrations

› Generate safe, version-controlled database migration scripts.

› Seamless schema updates with Flyway or Liquibase based on JPA modifications.

THANK YOU FOR ATTENDING

Let's Connect and Transform Java Development

Key Takeaways & Resources

- Agent Skills provide repeatable, context-aware workflows for modern Java development.
- Standardize workflows with portable, version-controlled `SKILL.md` files.
- Join the local developer ecosystem and the Java User Group Philippines to collaborate and share practices.



Connect with me!